

Blocks

1. Clock divider / counter (produces slow tick)
2. Debouncer (cleans button input)
3. Mode register (stores slow/fast state)
4. LED driver (selects which counter bit drives LED)

Step-by-step

1. **Write the RTL (top module)**
 - A free-running counter `ctr`.
 - `led <= ctr[24]` for slow, `led <= ctr[22]` for fast (pick bits based on your clock).
2. **Debounce the button**
 - Sample button into a small shift register or use a timer window (10–20 ms is typical).
 - Only accept a press when stable for the full window.
3. **Toggle a mode bit on a clean press event**
 - `if (pressed_pulse) mode <= ~mode;`
4. **Create constraints**
 - Map `clk`, `led`, `btn` to board pins (PCF/LPF). `nextpnr` documents PCF use for iCE40.
5. **Build**
 - iCE40: `yosys` → `nextpnr-ice40` → `icepack` (IceStorm provides Verilog-to-bitstream tools).
 - ECP5: `yosys` → `nextpnr-ecp5` (Trellis enables open tooling for ECP5).
6. **Program**
 - iCE40: `iceprog` is commonly used after `icepack` to load `.bin`.
7. **Debug checklist**
 - LED dead? Most likely: wrong pin, LED is active-low, or no clock pin.
 - Button flips twice? Your debounce window is too short or not edge-detecting.

Upgrades

- Add press-and-hold: long press resets mode.
- Add a second LED to show “button seen” pulses.

Portfolio packaging

- One GIF/video: slow ↔ fast toggle.
- A short README: explain bounce and your debounce method with a timing diagram reference.

Project 2: PWM controller (LED dimmer or fan-like control)

What you'll build

- PWM output on a pin (or LED) with adjustable duty cycle
- Button cycles duty: 10% → 30% → 60% → 90% → repeat
- Optional: smooth “breathing” demo

PWM is a perfect FPGA project because it's counter + compare, and it immediately looks like “control engineering.”

Blocks

1. PWM counter (e.g., 8–12 bits)
2. Duty register (stores duty value)
3. Comparator (`pwm_out = (ctr < duty)`)
4. UI (button cycles duty; reuse debounce from Project 1)

Step-by-step

1. **Pick your PWM resolution**
 - 8-bit PWM = 256 levels (fine for LED demo).
2. **Implement PWM**
 - Free-running counter, compare against duty.
3. **Cycle duty steps on button press**
 - Store duty in a small lookup: `{26, 77, 154, 230}` (≈10/30/60/90% of 255).
4. **Constrain pins**
 - Map `pwm_out` to LED pin (or a spare header pin if your board exposes one).
5. **Build + program** using the same open flow as Chapter 10.
6. **Debug**
 - If LED brightness barely changes: your PWM frequency is too high/low for the LED driver, or your LED is active-low (invert output).

Upgrades

- Add a second counter to create breathing: ramp duty up/down automatically.
- Add “thermal thinking”: cap max duty in fast mode.

Portfolio packaging

- Include a scope/logic-analyzer screenshot if you can. If not, show a waveform capture from simulation + a phone video of the LED dimming.

Project 3: UART serial monitor (your FPGA learns to talk)

What you'll build

- UART TX on a pin (or USB-UART bridge if your board has one)
- Prints a status byte every second (or on button press)
- Example output: `MODE=1 DUTY=154`

UART matters because it becomes your debug console for every later project. UART framing typically uses a start bit (low), data bits, optional parity, and stop bit(s); “8N1” is the common setting.

Blocks

1. Baud generator (clock divider)